



Sistema de Gestió de Nutrició i Entrenament Personalitzat Basat en Web

FitMeal – La teva app de fitness i nutrició

Autors: Kevin Castellón Ledezma · Tommy · [Nom 3]

Director: [Nom del director del projecte]

Curs: 2n DAW – 2025 / 2026

Centre: Centre d'Estudis Monlau

Data: Maig 2026



1. Dedicatòries i Agraïments

Volem dedicar aquest projecte a totes les persones que han estat al nostre costat durant aquests dos anys de formació. Als nostres professors, que han compartit amb nosaltres no tan sols coneixements tècnics sinó també la passió per la programació. Als companys de curs, per les hores de col·laboració, els debats i les solucions trobades en comú.

Agraïm especialment al nostre director de projecte per la seva orientació constant i per ajudar-nos a enfocar la complexitat tècnica del projecte. A les nostres famílies, per la paciència i el suport incondicional durant les nits de codi i els caps de setmana dedicats al desenvolupament.

Finalment, a la comunitat de codi obert i a totes les persones que han contribuït a les eines que hem fet servir: React, Node.js, MySQL, Tailwind CSS i tantes d'altres sense les quals aquest projecte no hauria estat possible.

2. Propòsit del Projecte

FitMeal neix de la necessitat cada cop més present de disposar d'una eina digital que integri, de manera cohesionada, la gestió de la nutrició i l'entrenament físic personal. En l'actualitat, la majoria d'aplicacions o bé se centren en la dieta o bé en l'exercici, però rarament ofereixen les dues dimensions en un únic entorn.

El projecte ens atrau per diverses raons. En primer lloc, és un domini real i amb impacte: qualsevol persona que vulgui millorar la seva salut necessita un pla nutricional i un entrenament adaptat als seus objectius. En segon lloc, tècnicament és un repte complet: requereix un back-end robust amb autenticació, gestió de rols, API RESTful i base de dades relacional, a més d'un front-end modern amb una experiència d'usuari cuidada.

A nivell personal, el projecte ens ha permès aplicar tots els coneixements adquirits durant el cicle en un context pràctic: des del disseny de la base de dades fins al desplegament en contenidors Docker, passant per la implementació de OAuth, el sistema de rols i la gestió d'uploads d'imatges. La possibilitat d'oferir una versió premium amb un entrenador personal virtual ha afegit una capa de complexitat que ha enriquit molt el projecte.



3. Índex

1	Dedicatòries i Agraïments	3
2	Propòsit del Projecte	3
3	Índex	4
4	Definició del Projecte	5
4.1	Requisits Funcionals i No Funcionals	5
4.2	Disseny Funcional	6
5	Estudi de Tecnologies i Eines	8
5.1	Back-end	8
5.2	Front-end	9
5.3	Base de Dades	10
5.4	Infraestructura i Desplegament	10
6	Implementació	11
6.1	Procés 1 – Autenticació i Gestió de Rols	11
6.2	Procés 2 – Gestió de Recetes i Nutrició	13
6.3	Procés 3 – Gestió d'Exercicis i Entrenament	15
6.4	Procés 4 – Perfil d'Usuari i Onboarding	17
6.5	Procés 5 – Botiga i Carret de la Compra	19
6.6	Procés 6 – Dashboard d'Administrador	21
6.7	Procés 7 – Dashboard d'Entrenador	22



6.8	Procés 8 – Sistema de Contacte	23
7	Millores i Línies de Futur	24
8	Conclusions	25
9	Bibliografia / Webgrafia	26
10	Annexos	27



4. Definició del Projecte

FitMeal és una aplicació web fullstack que permet als usuaris gestionar la seva alimentació i entrenament físic de forma personalitzada. La plataforma combina una API RESTful construïda amb Node.js/Express i una base de dades MySQL, amb un client web reactiu desenvolupat en React. El sistema diferencia quatre tipus d'usuaris: administradors, usuaris normals, usuaris premium i entrenadors personals.

4.1 Requisits Funcionals

Codi	Descripció
RF-01	El sistema ha de permetre el registre d'usuaris amb email/contrasenya o via OAuth Google.
RF-02	El sistema ha de permetre l'inici de sessió i la gestió de sessions amb JWT.
RF-03	L'usuari ha de poder completar un onboarding inicial per personalitzar el seu perfil.
RF-04	El sistema ha d'oferir un catàleg de recetes amb informació nutricional filtrable.
RF-05	L'usuari ha de poder afegir recetes i exercicis als seus favorits.
RF-06	El sistema ha d'oferir un catàleg d'exercicis organitzats per grup muscular i dificultat.
RF-07	Els usuaris premium han de poder ser assignats a un entrenador personal.
RF-08	L'entrenador ha de poder assignar i gestionar rutines per als seus clients.
RF-09	L'usuari ha de poder registrar el seu progrés d'entrenament (sèries, repeticions, pes).
RF-10	El sistema ha de disposar d'una botiga amb productes de fitness i un carret de la compra.
RF-11	L'administrador ha de poder gestionar (CRUD) usuaris, recetes i exercicis.
RF-12	El sistema ha de permetre l'enviament de missatges de contacte per correu electrònic.
RF-13	L'usuari ha de poder actualitzar el seu perfil i pujar una foto de perfil.
RF-14	El sistema ha d'implementar plans de subscripció (Bàsic, Avançat, Premium).

Requisits No Funcionals

Codi	Categoria	Descripció
RNF-01	Seguretat	Contrasenyes encriptades amb bcrypt. Tokens JWT amb expiració de 24h. Cookies httpOnly. Rate limiting en endpoints d'autenticació.
RNF-02	Rendiment	Pool de connexions a la base de dades (màx. 10). Paginació en llistats. Build



		optimitzat amb Vite.
RNF-03	Usabilitat	Interfície responsiva amb Tailwind CSS. Feedback visual amb toast notifikacions. Animacions fluides amb Framer Motion.
RNF-04	Mantenibilitat	Arquitectura MVC al back-end. Separació de preocupacions (context, components, pàgines). Variables d'entorn per a configuració.
RNF-05	Portabilitat	Contenidorització amb Docker i Docker Compose. Compatible amb Linux, Windows i macOS.
RNF-06	Escalabilitat	Disseny modular de l'API. Estructura preparada per afegir nous rols o funcionalitats.

4.2 Disseny Funcional – Arquitectura General

L'arquitectura del sistema segueix el patró client-servidor. El front-end és una SPA (Single Page Application) construïda amb React que es comunica amb el back-end a través d'una API RESTful. El back-end segueix el patró MVC (Model-View-Controller) amb Express, i persisteix les dades en una base de dades relacional MySQL.

[DIAGRAMA D'ARQUITECTURA: Client (React SPA) ↔ API REST (Node.js/Express) ↔ MySQL]

Diagrama de Casos d'Ús

El sistema identifica quatre actors principals: l'Usuari Anònim (visitant), l'Usuari Registrat (pla bàsic), l'Usuari Premium (amb accés a entrenador) i l'Administrador. A continuació es representen els principals casos d'ús:

[DIAGRAMA DE CASOS D'ÚS: Actors i Funcionalitats Principals]



Diagrama Entitat-Relació (E-R)

La base de dades conté les entitats principals i les seves relacions. A continuació es mostra el diagrama entitat-relació del sistema:

[DIAGRAMA E-R: Taules i Relacions de la Base de Dades]



5. Estudi de Tecnologies i Eines

Abans de iniciar la implementació, vam analitzar les alternatives disponibles per a cada capa de l'aplicació. La tria va estar guiada per criteris tècnics com la maduresa de la tecnologia, la seva adopció en l'indústria, la corba d'aprenentatge i la compatibilitat entre components.

5.1 Back-end – Node.js + Express

Per al servidor, es van avaluar tres opcions principals: PHP/Laravel, Python/Django i Node.js/Express. La tria final va recaure en Node.js per les següents raons:

- JavaScript al back-end i front-end redueix el canvi de context i unifica l'ecosistema.
- Express és minimalista i flexible, ideal per construir una API RESTful a mida.
- L'ecosistema npm ofereix una gran varietat de paquets madurs (passport, multer, nodemailer).
- Node.js té molt bona adopció professional, la qual cosa facilita la recerca de documentació.
- El model no-bloquejant és eficient per gestionar múltiples peticions concurrents.

Paquet	Funció
Express 5.1.0	Framework web minimalista per a Node.js
mysql2 3.15.3	Driver MySQL amb suport promises i pool de connexions
jsonwebtoken 9.0.2	Generació i verificació de tokens JWT
passport.js	Middleware d'autenticació (estratègia Google OAuth 2.0)
bcryptjs	Hash i verificació de contrasenyes
multer	Gestió d'uploads de fitxers (fotos de perfil)
nodemailer	Enviament d'emails (formulari de contacte)
helmet	Capçaleres de seguretat HTTP
express-rate-limit	Limitació de peticions per IP
swagger-ui-express	Documentació automàtica de l'API
dotenv	Gestió de variables d'entorn

5.2 Front-end – React 19 + Vite + Tailwind CSS

Per al client web, es van comparar Vue 3, Angular i React. La decisió va ser React per la seva flexibilitat, la gran comunitat, la compatibilitat amb Three.js per als elements 3D i per la robustesa de React Router per a la navegació SPA.

Paquet	Funció
React 19.2.0	Biblioteca per a interfícies d'usuari basada en components
React Router DOM 7	Enrutament declaratiu per a SPA
Vite 7.3.1	Bundler ultraràpid amb HMR (Hot Module Replacement)
Tailwind CSS 4.1.18	Framework CSS utility-first per a disseny responsiu
Axios 1.13.5	Client HTTP per a peticions a l'API
Framer Motion 12.35	Biblioteca d'animacions declaratives
React-hot-toast 2.6	Notificacions tipus toast
Recharts 3.8.1	Gràfics de progrés i estadístiques



5.3 Base de Dades – MySQL 8.0

Es va triar MySQL per la seva fiabilitat, el suport a transaccions ACID, la compatibilitat amb hosting compartit i la familiaritat de l'equip. Es va descartar PostgreSQL per no aportar avantatges decisius per al nostre cas d'ús, i MongoDB per la naturalesa relacional de les dades (usuaris, recetes, exercicis, rols).

5.4 Infraestructura – Docker + Docker Compose

L'ús de Docker permet garantir que l'entorn de desenvolupament sigui idèntic al de producció. Docker Compose orquestra els dos serveis principals: el servidor Node.js i la base de dades MySQL.

```
# docker-compose.yml (esquema)
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: fitmeal
  backend:
    build: .
    depends_on: [db]
    ports: ['3000:3000']
    env_file: .env
```



6. Implementació

La implementació s'ha estructurat en vuit processos que cobreixen les funcionalitats principals de l'aplicació. Per a cadascun s'especifica l'objectiu, les possibles solucions estudiades, els riscos identificats, la implementació realitzada i les proves unitàries dutes a terme.

6.1 Procés 1 – Autenticació i Gestió de Rols

a) Objectiu

Implementar un sistema d'autenticació segur que permeti el registre i inici de sessió amb email/contrasenya i via OAuth 2.0 de Google. Gestionar quatre rols d'usuari (Admin, Normal, Premium, Entrenador) amb protecció de rutes tant al front-end com al back-end.

b) Estudi de Solucions

Solució	Avaluació
Sessions + Cookies	Senzill, però dificulta l'escalabilitat horitzontal i les APIs stateless.
JWT + localStorage	Fàcil implementació, però vulnerable a XSS.
JWT + httpOnly Cookie	✅ Triat. Combina la simplicitat de JWT amb la seguretat de les httpOnly cookies.
OAuth únicament	No permet registre propi. Dependència de tercers.

c) Riscos

- Robatori de tokens si s'usen cookies SameSite mal configurades.
- Atacs de força bruta sobre el formulari de login.
- Tokens caducats que no es redirigeixen correctament.
- Configuració incorrecta del callback OAuth en producció.

d) Implementació

El sistema utilitza JWT amb una expiració de 24 hores. Les credencials de l'usuari es validen al back-end i el token es retorna en una httpOnly cookie (no accessible des de JavaScript). El middleware auth.js verifica el token en cada petició protegida i comprova el rol necessari per accedir al recurs.

```
// middleware/auth.js - verifyToken
const verifyToken = (req, res, next) => {
  const token = req.cookies.token || req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token provided' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid token' });
  }
};
```



```
// Protecció per rol
const requireRole = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.id_rol)) {
    return res.status(403).json({ error: 'Insufficient permissions' });
  }
  next();
};
```

[CAPTURA: Pàgina de Login i Pàgina de Registre]

[CAPTURA: Flux OAuth Google – Pantalla de Selecció de Compte]

e) Proves Unitàries

Test	Descripció	Resultat Esperat	Estat
T-AUTH-01	Registre amb email vàlid	Retorna 201 + JWT cookie	✓ Passa
T-AUTH-02	Login amb credencials incorrectes	Retorna 401	✓ Passa
T-AUTH-03	Accés a ruta protegida sense token	Retorna 401	✓ Passa
T-AUTH-04	Accés a ruta d'admin amb rol=2	Retorna 403	✓ Passa
T-AUTH-05	Rate limit: >10 intents en 15min	Retorna 429	✓ Passa
T-AUTH-06	Login amb Google OAuth	Redirigeix correctament	✓ Passa



6.2 Procés 2 – Gestió de Recetes i Nutrició

a) Objectiu

Proporcionar un catàleg de recetes amb informació nutricional completa (calories, proteïnes, carbohidrats, grasses, temps de preparació). Permetre als usuaris cercar, filtrar i guardar recetes als seus favorits. L'administrador ha de poder gestionar el catàleg complet.

b) Estudi de Solucions

Es va valorar l'ús d'una API externa de nutrició com Edamam o USDA FoodData Central per obtenir dades nutricionals automàticament. No obstant, es va decidir mantenir una base de dades pròpia per tenir control total sobre el contingut i evitar dependències externes que podrien afectar la disponibilitat o el cost del servei.

c) Riscos

- Dades nutricionals inexactes si s'introdueixen manualment.
- Càrrega excessiva de la pàgina si es recuperen totes les recetes sense paginació.
- Imatges d'alta resolució que alenteixen la càrrega inicial.

d) Implementació

La taula recetes conté: id_receta, titulo, calorías, proteína, carbohidratos, grasas, tiempo, tipo, imagen, ingredientes i instrucciones. L'API exposa endpoints CRUD protegits per rol d'administrador. El front-end implementa filtratge per tipus (breakfast, lunch, dinner, snack) i cercador per títol.

[CAPTURA: Pàgina de Llistat de Recetes amb Filtres]

[CAPTURA: Pàgina de Detall de Receita (Ingredients + Valors Nutricionals)]

e) Proves

Test	Acció	Resultat Esperat	Estat
T-REC-01	GET /api/recipes	Retorna array de recetes	✓
T-REC-02	GET /api/recipes/:id	Retorna recepta per ID	✓



T-REC-03	POST /api/recipes sense token admin	Retorna 403	✓
T-REC-04	Filtrat per tipus 'breakfast'	Llista filtrada correctament	✓
T-REC-05	Afegir/eliminar favorit	BD actualitzada correctament	✓



6.3 Procés 3 – Gestió d'Exercicis i Entrenament

a) Objectiu

Oferir un catàleg d'exercicis organitzats per grup muscular amb nivell de dificultat, descripció i punts clau. Permetre el seguiment del progrés (sèries, repeticions, pes) i la visualització de models anatòmics 3D per identificar els grups musculars.

b) Estudi de Solucions

Per a la visualització anatòmica es va valorar l'ús d'imatges estàtiques vs. un model 3D interactiu. Es va triar Three.js + React Three Fiber per oferir una experiència visual diferencial i educativa que permet a l'usuari identificar exactament quins músculs treballa cada exercici.

d) Implementació

La taula ejercicios conté: id, musculo_id, titulo, dificultad, imagen, descripcion, tipo i puntos_clave. L'endpoint GET /api/exercises/:muscleName filtra per grup muscular. El sistema de progrés emmagatzema date, series, repeticiones i peso per usuari i exercici.

[CAPTURA: Pàgina d'Exercicis – Selecció per Grup Muscular]

[CAPTURA: Model 3D Anatòmic + Detall d'Exercici]

[CAPTURA: Formulari de Registre de Progrés]

e) Proves

Test	Acció	Resultat Esperat	Estat
------	-------	------------------	-------



T-EX-01	GET /api/exercises	Llista completa d'exercicis	✓
T-EX-02	GET /api/exercises/pectorals	Filtra per múscul	✓
T-EX-03	POST /api/progress-exercises	Registra progrés correctament	✓
T-EX-04	GET /api/favorites-exercises	Llista favorits de l'usuari	✓



6.4 Procés 4 – Perfil d'Usuari i Onboarding

a) Objectiu

Recollir les dades inicials de l'usuari (pes, alçada, gènere, objectiu, nivell d'activitat, preferències alimentàries) a través d'un onboarding guiat per calcular automàticament les macros recomanades. Permetre l'actualització del perfil i la pujada de foto.

b) Estudi de Solucions

Es va valorar demanar les dades en el moment del registre o en un onboarding posterior. Es va triar el flux d'onboarding post-registre per no allargar el formulari inicial i reduir l'abandonament durant el registre. L'onboarding es marca com a complet (onboarding_completado=1) i no es torna a mostrar.

d) Implementació

La calculadora de macros utilitza la fórmula de Mifflin-St Jeor per calcular el TMB (Taxa Metabòlica Basal) i aplica el factor d'activitat per obtenir les calories totals diàries. Distribueix les macros segons l'objectiu: pèrdua de pes, manteniment o guany muscular.

```
// utils/macrosCalculator.js
export function calculateMacros(weight, height, age, gender, activityLevel, goal)
{
  // Mifflin-St Jeor BMR
  const bmr = gender === 'male'
    ? 10 * weight + 6.25 * height - 5 * age + 5
    : 10 * weight + 6.25 * height - 5 * age - 161;
  const activityFactors = { sedentary:1.2, light:1.375, moderate:1.55,
    active:1.725 };
  const tdee = bmr * activityFactors[activityLevel];
  // Goal adjustments
  const calories = goal === 'lose' ? tdee - 500 : goal === 'gain' ? tdee + 300 :
    tdee;
  return { calories, protein: weight * 2, carbs: ..., fat: ... };
}
```

[CAPTURA: Formulari d'Onboarding Pas a Pas]



[CAPTURA: Pàgina de Perfil amb Dades i Foto]



6.5 Procés 5 – Botiga i Carret de la Compra

a) Objectiu

Implementar una botiga de productes de fitness (suplements, roba, equipament) amb carret de la compra, gestió de quantitats i talla, i procés de checkout. La persistència del carret es gestiona localment per simplificar la implementació.

b) Estudi de Solucions

Solució	Avaluació
Carret al backend (BD)	Persistència entre dispositius, però requereix més endpoints i complexitat.
Carret a localStorage	✅ Triat. Senzill, ràpid, sense peticions al servidor. Suficient per MVP.
Carret a Redux/Zustand	Interessant per apps grans, però innecessari per a la nostra escala.

d) Implementació

CartContext.jsx gestiona l'estat global del carret mitjançant useReducer i localStorage. Cada producte s'identifica per id + talla per permetre la mateixa referència en múltiples talles. El checkout mostra un formulari d'adreça de lliurament (simulat, sense passerella de pagament real).

[CAPTURA: Pàgina de Productes de la Botiga]

[CAPTURA: Pàgina de Detall de Producte]

[CAPTURA: Carret de la Compra i Checkout]





6.6 Procés 6 – Dashboard d'Administrador

a) Objectiu

Proporcionar una interfície d'administració per gestionar tots els recursos del sistema: usuaris, recetes i exercicis. Mostrar estadístiques generals de la plataforma.

d) Implementació

El dashboard d'administrador és accessible exclusivament per a usuaris amb rol=1. Utilitza AdminProtectedRoute per redirigir als usuaris sense permís. L'API exposa GET /api/admin/stats que retorna totals de cada entitat.

Endpoint	Funció
GET /api/admin/stats	Estadístiques: total usuaris, recetes, exercicis, comandes
GET /api/admin/users	Llistat paginat d'usuaris amb filtres
GET /api/admin/recipes	Llistat de recetes amb opcions CRUD
GET /api/admin/exercises	Llistat d'exercicis amb opcions CRUD

[CAPTURA: Dashboard d'Admin – Estadístiques Globals]

[CAPTURA: Admin – Gestió d'Usuaris (Llistat + Accions)]

[CAPTURA: Admin – Gestió de Recetes i Exercicis]



6.7 Procés 7 – Dashboard d'Entrenador

a) Objectiu

Permetre als usuaris amb rol d'entrenador (rol=4) gestionar els seus clients: veure la llista de clients assignats, crear i assignar rutines d'exercicis personalitzades, registrar l'entrenament realitzat i fer el seguiment del progrés diari.

d) Implementació

La taula `entrenador_cliente` relaciona entrenadors i clients amb estat (activo/inactivo) i data d'assignació. L'entrenador pot veure la rutina de cada client i afegir exercicis des del catàleg. L'endpoint POST `/api/trainers/log-workout` registra el treball realitzat per data.

[CAPTURA: Dashboard d'Entrenador – Llista de Clients]

[CAPTURA: Vista de Rutina d'un Client + Assignació d'Exercicis]



6.8 Procés 8 – Sistema de Contacte per Email

a) Objectiu

Implementar un formulari de contacte que envii un email real a l'equip de FitMeal i una confirmació automàtica a l'usuari. El sistema no requereix autenticació per facilitar el contacte de nous usuaris potencials.

d) Implementació

S'utilitza Nodemailer configurat amb Gmail SMTP. Quan un usuari envia el formulari, el back-end genera dos correus: un per l'equip intern amb les dades del contacte, i un altre de confirmació per a l'usuari que ha enviat el missatge.

```
// Exemple configuració Nodemailer
const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: process.env.GMAIL_USER,
    pass: process.env.GMAIL_APP_PASSWORD // App password, no la contrasenya real
  }
});

// POST /api/contact
await transporter.sendMail({
  from: process.env.GMAIL_USER,
  to: 'fitmeal@gmail.com',
  subject: `Nou contacte de ${nombre}`,
  html: `<p>${mensaje}</p>`
});
```

[CAPTURA: Pàgina de Contacte amb Formulari]

[CAPTURA: Email de Confirmació Rebut per l'Usuari]





Estructura de la Base de Dades

A continuació es detalla l'estructura de les taules principals de la base de dades MySQL. El sistema utilitza un pool de connexions amb un màxim de 10 connexions concurrents.

[CAPTURA: Esquema SQL – Vista del Workbench o phpMyAdmin]

Taula: usuarios

Camp	Tipus / Descripció
id_usuario	INT PK AUTO_INCREMENT
email	VARCHAR(100) UNIQUE NOT NULL
password_hash	VARCHAR(255)
nombre	VARCHAR(100)
apellidos	VARCHAR(100)
id_rol	INT FK → roles(id_rol)
plan	ENUM('basico','avanzado','premium')
peso	DECIMAL(5,2)
altura	INT
genero	ENUM('male','female','other')
nivel_actividad	VARCHAR(50)
objetivo	VARCHAR(100)
onboarding_completado	TINYINT(1) DEFAULT 0
foto_url	VARCHAR(255)

Taula: recetas

Camp	Tipus / Descripció
id_receta	INT PK AUTO_INCREMENT
titulo	VARCHAR(200) NOT NULL
calorias	INT
proteina	DECIMAL(6,2)
carbohidratos	DECIMAL(6,2)
grasas	DECIMAL(6,2)
tiempo	INT (minuts)
tipo	VARCHAR(50)
imagen	VARCHAR(255)
ingredientes	TEXT (JSON)
instrucciones	TEXT

Taula: ejercicios

Camp	Tipus / Descripció
------	--------------------



id	INT PK AUTO_INCREMENT
musculo_id	INT FK → musculos
titulo	VARCHAR(200)
dificultad	ENUM('Baja','Media','Alta')
imagen	VARCHAR(255)
descripcion	TEXT
tipo	VARCHAR(50)
puntos_clave	TEXT (JSON)

Taula: productos

Camp	Tipus / Descripció
id_producto	INT PK AUTO_INCREMENT
nombre_producto	VARCHAR(200)
descripcion	TEXT
precio	DECIMAL(10,2)
stock	INT
id_categoria	INT FK → categorias_productos
imagen_url	VARCHAR(255)
estado	ENUM('activo','inactivo')

Taula: entrenador_cliente

Camp	Tipus / Descripció
id_entrenador	INT FK → usuarios
id_cliente	INT FK → usuarios
estado	ENUM('activo','inactivo')
fecha_asignacion	DATETIME

Taula: progreso_ejercicios

Camp	Tipus / Descripció
id	INT PK AUTO_INCREMENT
id_usuario	INT FK → usuarios
id_ejercicio	INT FK → ejercicios
fecha	DATE
series	INT
repeticiones	INT
peso	DECIMAL(5,2)



Diagrames de Flux dels Processos Principals

Flux d'Autenticació

```
Usuari omple formulari de login
→ POST /auth/login
→ authController.login() verifica email+password (bcrypt.compare)
→ Si OK: generateToken() → JWT a httpOnly cookie
  → AuthContext.login() guarda user + token a context
  → React Router redirigeix a /perfil o /onboarding
→ Si KO: res.status(401) → toast d'error al formulari

Google OAuth:
Usuari clica 'Login amb Google'
→ GET /auth/google → Passport redirigeix a Google
→ Google autentica i torna al callback
→ GET /auth/google/callback
→ Passport crea/troba usuari a BD
→ JWT cookie → redirigeix al frontend
```

[DIAGRAMA: Flux d'Autenticació JWT + OAuth]

Flux de Creació d'una Rutina d'Entrenament

```
Entrenador accedeix a /entrenador
→ GET /api/trainers/clients → Llista clients
→ Selecciona client → GET /api/trainers/clients/:id/routine
→ Veu rutina actual del client
→ Selecciona exercici del catàleg
→ POST /api/trainers/assign-exercise
→ DB: INSERT a rutina_ejercicios
→ Client veu rutina actualitzada a /rutina
```

[DIAGRAMA: Flux de Gestió de Rutina Entrenador-Client]



Flux de Compra a la Botiga

```
Usuari visita /products
→ GET /api/products → Llistat de productes
→ Clica producte → /products/:id
→ Selecciona talla + quantitat
→ addToCart() → CartContext + localStorage
→ Icona carret actualitza badge
→ Usuari va a /cart → Revisa items
→ Continua a /checkout
→ Omple adreça → Confirma (simulat)
```

[DIAGRAMA: Flux de Compra a la Botiga]



7. Millores i Línies de Futur

Si Comencéssim de Nou...

Àrea	Millora Proposada
Passerella de pagament real	Integraríem Stripe o PayPal des del primer dia per al checkout, evitant la solució simulada actual.
TypeScript al back-end	Hauríem usat TypeScript des d'inici per evitar errors de tipus i millorar el manteniment.
ORM (Prisma / Sequelize)	En lloc de SQL raw, un ORM hauria simplificat les consultes i les migracions.
Testing des del dia 1	Hauríem implementat tests unitaris i d'integració amb Jest des de l'inici, no al final.
Disseny mòbil primer	Algunes pàgines es van dissenyar primer per a escriptori; el mòbil-first hauria evitat retalls posteriors.
Variables d'entorn unificades	Separar millor els entorns dev/staging/prod amb fitxers .env específics.

Si Seguíssim Treballant...

Funcionalitat	Descripció
Passerella de pagament	Integrar Stripe per a pagaments reals a la botiga i subscripcions premium.
App mòbil	Desenvolupar una aplicació nativa amb React Native reutilitzant la lògica de negoci existent.
IA nutricional	Integrar un model de llenguatge per generar plans de dieta personalitzats automàticament.
Tracking avançat	Gràfics de progrés temporal del pes, composició corporal i records personals.
Notificacions push	Sistema de recordatoris per a sessions d'entrenament i ingesta hídrica.
Social features	Perfils públics, reptes entre usuaris, sistema de punts i badges.
Integració amb wearables	Connexió amb Google Fit, Apple Health i dispositius com Garmin o Fitbit.
Videotrucades entrenador	Integrar WebRTC per a sessions en viu entre entrenador i client.
CI/CD complet	Pipelines automatitzats amb GitHub Actions, tests automàtics i desplegament continu.



8. Conclusions

El projecte FitMeal ha suposat un repte tècnic i personal de gran envergadura que ha permès consolidar i posar en pràctica de forma integrada tots els coneixements adquirits durant el cicle formatiu de Desenvolupament d'Aplicacions Web.

Des del punt de vista tècnic, el projecte ha cobert un espectrum molt ampli de competències: disseny de bases de dades relacionals, implementació d'una API RESTful amb autenticació JWT i OAuth, gestió d'arxius, enviament de correus electrònics reals, contenidorització amb Docker, i desenvolupament d'una SPA completa amb React que inclou animacions, gràfics, models 3D i un sistema de rols.

Un dels aprenentatges més valuosos ha estat la gestió de la complexitat creixent d'un projecte real. A diferència dels exercicis acadèmics, FitMeal ha requerit decisions d'arquitectura que tinguessin en compte el manteniment futur, la seguretat i l'experiència d'usuari de forma simultània. Hem après que no existeix una solució perfecta: sempre hi ha compromisos entre simplicitat, rendiment i funcionalitat.

El treball en equip ha estat fonamental. La coordinació a través de Git (gestió de branques, resolució de conflictes, pull requests) ha millorat substancialment les nostres habilitats de col·laboració professional. Hem après a comunicar decisions tècniques, dividir tasques de forma efectiva i resoldre bloquejos de forma conjunta.

Respecte a les limitacions actuals, som conscients que el checkout és simulat, que els tests automatitzats haurien d'estar millor coberts i que alguns aspectes del disseny mòbil es podrien polir. Tanmateix, estem satisfets d'haver construït una aplicació funcional, segura i amb una base sòlida sobre la qual continuar construint.

En resum, FitMeal és molt més que un projecte de fi de cicle: és una demostració pràctica de la nostra capacitat per afrontar el desenvolupament d'una aplicació web real, des de l'especificació fins al desplegament, amb les eines i metodologies que utilitza la indústria avui en dia.



9. Bibliografia / Webgrafia

Ref.	Recurs	Enllaç / Referència
[1]	MDN Web Docs – JavaScript Reference	https://developer.mozilla.org
[2]	React Documentation	https://react.dev
[3]	Express.js Documentation	https://expressjs.com
[4]	Node.js Documentation	https://nodejs.org/en/docs
[5]	MySQL 8.0 Reference Manual	https://dev.mysql.com/doc/refman/8.0/en/
[6]	Tailwind CSS Documentation	https://tailwindcss.com/docs
[7]	Passport.js Documentation	https://www.passportjs.org/docs/
[8]	JSON Web Tokens (JWT) – RFC 7519	https://datatracker.ietf.org/doc/html/rfc7519
[9]	Docker Documentation	https://docs.docker.com
[10]	Vite Documentation	https://vitejs.dev/guide/
[11]	React Three Fiber Documentation	https://docs.pmnd.rs/react-three-fiber
[12]	Framer Motion Documentation	https://www.framer.com/motion/
[13]	OWASP Top 10 – Security Risks	https://owasp.org/www-project-top-ten/
[14]	Mifflin-St Jeor BMR Equation	American Journal of Clinical Nutrition, 1990
[15]	Nodemailer Documentation	https://nodemailer.com/about/
[16]	Recharts Documentation	https://recharts.org/en-US/api
[17]	Swagger / OpenAPI Specification	https://swagger.io/specification/



10. Annexos

Annex A – Estructura de Fitxers del Projecte

```
FitMealProjecto/
├── config/
│   ├── database.js          # Pool connexions MySQL
│   └── passport.js          # Estratègia OAuth Google
├── controllers/
│   ├── authController.js    # Login, register, verifyToken
│   ├── userController.js    # CRUD usuaris
│   ├── recipeController.js  # CRUD recetes
│   ├── exerciseController.js
│   ├── productController.js
│   ├── trainerController.js # Lògica entrenador-client
│   └── adminController.js   # Estadístiques admin
├── middleware/
│   ├── auth.js              # verifyToken, requireRole
│   ├── upload.js            # Multer (fotos perfil)
│   └── log.js                # Logger de peticions
├── models/                  # Funcions de consulta BD
├── routes/                  # Definició endpoints
├── frontend/
│   └── src/
│       ├── api/axios.js     # Client HTTP configurat
│       ├── context/         # AuthContext, CartContext
│       ├── pages/           # Pàgines (Login, Home, Perfil...)
│       ├── components/      # Navbar, Footer, ProtectedRoute
│       └── utils/           # macrosCalculator.js
├── index.js                 # Punt d'entrada servidor
├── docker-compose.yml
└── .env.example
```

Annex B – Variables d'Entorn Necessàries (.env)

```
# Base de dades
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=****
DB_NAME=fitmeal
DB_PORT=3306

# Servidor
PORT=3000
NODE_ENV=development

# JWT i Sessions
JWT_SECRET=clau_secret_a_molt_llarga
SESSION_SECRET=clau_sessio
```



```
# OAuth Google
GOOGLE_CLIENT_ID=xxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=xxx
GOOGLE_CALLBACK_URL=http://localhost:3000/auth/google/callback

# Email (Nodemailer)
GMAIL_USER=fitmeal@gmail.com
GMAIL_APP_PASSWORD=xxxx_xxxx_xxxx

# CORS
FRONTEND_URL=http://localhost:5173
ALLOWED_ORIGINS=http://localhost:5173,http://localhost:3000
```

Annex C – Endpoints de l'API REST (Resum)

Mètode	Endpoint	Auth	Descripció
POST	/auth/register	No	Registrar nou usuari
POST	/auth/login	No	Inici de sessió
GET	/auth/verify	JWT	Verificar token actiu
GET	/auth/google	No	Iniciar OAuth Google
GET	/api/recipes	No	Llistat de recetes
GET	/api/recipes/:id	No	Detall recepta
POST	/api/recipes	Admin	Crear recepta
PUT	/api/recipes/:id	Admin	Actualitzar recepta
DELETE	/api/recipes/:id	Admin	Eliminar recepta
GET	/api/exercises	No	Llistat exercicis
GET	/api/exercises/:muscleName	No	Filtrar per múscul
GET	/api/products	No	Llistat productes
GET	/api/favorites	JWT	Recetes favorites
POST	/api/favorites	JWT	Afegir favorita
GET	/api/progress-exercises	JWT	Progrés entrenaments
POST	/api/progress-exercises	JWT	Registrar progrés
GET	/api/trainers/clients	Entrenador	Clients assignats
POST	/api/trainers/assign-exercise	Entrenador	Assignar exercici
GET	/api/admin/stats	Admin	Estadístiques globals
POST	/api/contact	No	Enviar email contacte

Annex D – Captures de Pantalla del Projecte

[CAPTURA: Pàgina Principal (Home) – Hero Section]



[CAPTURA: Pàgina Principal – Secció de Plans de Subscripció]

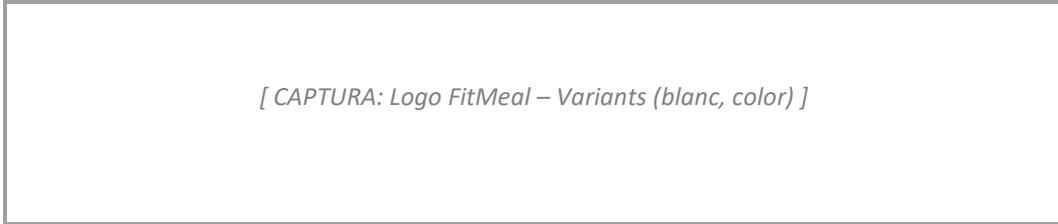
[CAPTURA: Pàgina Principal – Secció de Funcionalitats]

[CAPTURA: Login – Versió Escriptori]

[CAPTURA: Login – Versió Mòbil (Responsiu)]

[CAPTURA: Dashboard d'Administrador complet]

[CAPTURA: Pàgina de Perfil d'Usuari completa]



[CAPTURA: Logo FitMeal – Variants (blanc, color)]